

Sommaire

1 Modules

```
1  # Importation de modules auxiliaires de Python
2  import random as rd
3  import time
4  import threading
5  import socket
6  import sys
7  # Module utilisé pour l'utilisation d'une fonction de hachage
8  import hashlib
9  # Modules nécessaires pour l'implémentation d'AES avec des bibliothèques
10 from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
11 from cryptography.hazmat.backends import default_backend
12 from cryptography.hazmat.primitives import padding
13 # Importation d'autres fichiers Python
14 import fonctions_utiles as fu
15 import algos_chiffrement as algos
```

Diapositive 36

2 algos_chiffrement.py

```
1 def aux_modulo(x, n, m, acc_reste):
2     """Calcul de  $x^n \bmod m$  avec la méthode d'exponentiation rapide"""
3     if n == 1:
4         return acc_reste % m
5     elif n % 2 == 0:
6         return aux_modulo(x*x % m, int(n/2), m, acc_reste*x % m)
7     else:
8         return aux_modulo(x*x % m, int((n-1)/2), m, acc_reste*x*x % m)
9
10 def modulo_exp(x:int, n:int, m:int):
11     """Fonction qui calcule  $x^n \bmod m$ """
12     if n == 0:
13         return 1 % m
14     else:
15         return aux_modulo(x, n, m, x)
16
17 def string_to_int(chaine: str, base=27):
18     """Fonction qui convertit une chaîne de caractères en entier et renvoie cet entier"""
19     n = len(chaine)
20     entier = 0
21
22     for k in range(1, n+1):
23         entier += dico_char_a_int[chaine[k-1]] * base**(n - k)
24     return entier
```

Diapositive 37

```
1 def int_to_string(entier: int, taille_mot: int, b=27):
2     """Fonction qui convertit un entier en chaîne de caractères et renvoie cette chaîne"""
3
4     quotient = 0
5     diviseur = 0
6     dividende = entier
```

```

7     mot = ""
8
9     for i in range(taille_mot - 1, 0, -1):
10         diviseur = b**i
11         quotient = dividende // diviseur
12         mot += liste_int_a_char[quotient]
13         dividende -= quotient * diviseur
14     mot += liste_int_a_char[dividende%b]
15
16     return mot
17
18 # Création de l'exception "InvNotFound" utile dans la suite
19 class InvNotFound(Exception):
20     pass

```

Diapositive 38

```

1 def inverse_mod(b: int, n: int):
2     """_Fonction_ qui_ prend_ en_ paramètre_ deux_ entiers_ b_ et_ n_ et_ renvoie_ l'_
3         inverse_ de_ b_ modulo_ n_ (ie_ b^(-1))
4         ou_ bien_ renvoie_ l'_erreur_ 'InvNotFound' _si_ b^(-1) _n'_existe_ pas_ """
5     n_0 = n
6     b_0 = b
7     t_0 = 0
8     t = 1
9     q = int(n_0/b_0)
10    r = n_0 - q * b_0
11    while r > 0:
12        temp = t_0 - q * t
13        if temp >= 0:
14            temp = temp % n
15        else:
16            temp = n - ((-temp) % n)
17        t_0 = t
18        t = temp
19        n_0 = b_0
20        b_0 = r
21        q = int(n_0/b_0)
22        r = n_0 - q * b_0
23    if b_0 != 1:
24        raise InvNotFound()
25    else:
26        return t

```

Diapositive 39

```

1 def creation_cles(lst_nb_prem: list):
2     """_Fonction_ prenant_ en_ paramètre_ une_ liste_ de_ nombres_ premiers_ et_ qui_ crée_
3         puis_ renvoie_ un_ couple_ de_ la_ forme_
4         (cle_publique, cle_privée) _en_ suivant_ la_ méthode_ de_ création_ des_ clés_ de_ l'_
5         'algorithme_ de_ chiffrement_ RSA'
6         """
7     e = 2
8     # On choisit aléatoirement les valeurs de p et q distinctes
9     p = lst_nb_prem.pop(rd.randint(0, len(lst_nb_prem) - 1))
10    q = rd.choice(lst_nb_prem)
11    # On choisit e et phi_n

```

```

11     n = p * q
12     phi_n = (p - 1) * (q - 1)
13
14     for i in range(2, phi_n): # on cherche le premier e qui convient en partant
        de e = 2
15         if fu.pgcd(i, phi_n) == 1:
16             e = i
17             break
18     d = inverse_mod(e, phi_n)
19
20     return ((n, e), d) # on renvoie (clé publique, clé privée)

```

Diapositive 40

```

1 def chiffrer_RSA_naif(m: int, cle_pb: tuple):
2     """Fonction qui prend en paramètre un entier m qui correspond au message à
        chiffrier et un tuple cle_pb qui
3     correspond à la clé de chiffrement. Elle renvoie un entier correspondant au
        message chiffré"""
4
5     assert (m < cle_pb[0])
6     # print("début calcul chiffrement et cle_pb : ", cle_pb)
7     reste = 1
8     for _ in range(cle_pb[1]):
9         reste *= m
10        reste = reste % cle_pb[0]
11    return reste
12
13 def chiffrer_RSA_v2(m: int, cle_pb: tuple):
14     """Fonction qui prend en paramètre un entier m qui correspond au message à
        chiffrier et un tuple cle_pb qui
15    correspond à la clé de chiffrement. Elle renvoie un entier correspondant au
        message chiffré"""
16
17    assert (m < cle_pb[0])
18    return modulo_exp(m, cle_pb[1], cle_pb[0])

```

Diapositive 41

```

1 def dechiffrer_RSA_naif(x: int, cle_pv: int, n: int):
2     """Fonction prenant en paramètre un entier x correspondant au message à dé
        chiffrier, cle_pv un entier
3     correspondant à la clé de déchiffrement et n issue de la clé de chiffrement
        du message. Elle renvoie un entier
4     correspondant au message déchiffré"""
5
6     reste = 1
7     for _ in range(cle_pv):
8         reste *= x
9         reste = reste % n
10
11    return reste
12
13 def dechiffrer_RSA_v2(x: int, cle_pv: int, n: int):
14     """Fonction prenant en paramètre un entier x correspondant au message à dé
        chiffrier, cle_pv un entier
15    correspondant à la clé de déchiffrement et n issue de la clé de chiffrement
        du message. Elle renvoie un entier

```

```

16     correspondant_au_message_déchiffré"""
17
18     return modulo_exp(x, cle_pv, n)

```

Diapositive 42

```

1  def chiffrer_chaine_RSA(chaine:str, cle_pb:tuple, taille_facteur=-1):
2      """_Fonction qui prend en paramètre une chaîne de caractères 'chaine' et un
        tuple 'cle_pb' et renvoie
3      une liste d'entiers correspondants aux facteurs de taille 'taille_facteur',
        chiffrés avec la fonction 'chiffrer_RSA(...)',
4      dont la concaténation forme la chaîne 'chaine' avant son chiffrement"""
5      if taille_facteur == (-1):
6          taille_facteur = len(chaine)
7          chaine_chiffree = []
8          chaine_fractionnee = []
9          chaine_courante = ""
10
11     for i in range(len(chaine)): # On décompose 'chaine' en une liste de
        facteurs de taille 'taille_facteur'
12         chaine_courante += chaine[i]
13         if len(chaine_courante) % taille_facteur == 0:
14             chaine_fractionnee.append(chaine_courante)
15             chaine_courante = ""
16
17     if chaine_courante != "":
18         for i in range(len(chaine_courante), taille_facteur): # on ajoute des
            espaces pour que le dernier facteur soit de taille taille_facteur
19             chaine_courante += " "
20             chaine_fractionnee.append(chaine_courante) # on ajoute le dernier
            facteur de taille < 'taille_facteur' (sans compter les espaces)
21
22     for facteur in chaine_fractionnee: # On chiffre tous les facteurs avec la
        fonction 'chiffrer_RSA(...)'
23         int_facteur_courant = string_to_int(facteur, base=95)
24         chaine_chiffree.append(chiffrer_RSA_v2(int_facteur_courant, cle_pb))
25     return chaine_chiffree

```

Diapositive 43

```

1  def dechiffrer_chaine_RSA(liste:list, cle_pv: int, n: int, taille_entiers=1):
2      """_Fonction qui prend en paramètre une liste d'entiers 'liste', un entier
        'cle_pv' et un entier n et
3      renvoie la concaténation des chaînes de caractères déchiffrées avec
        dechiffrer_RSA(...)' à partir des entiers
4      contenu dans 'liste' représentant des chaînes de caractères de taille
        'taille_entiers'"""
5
6      chaine_dechiffree = ""
7
8      for i in range(len(liste)): # On déchiffre tous les termes de liste sauf le
        dernier
9          chaine_courante = int_to_string(dechiffrer_RSA_v2(liste[i], cle_pv, n),
            taille_entiers, b=95)
10         chaine_dechiffree += chaine_courante
11
12     return chaine_dechiffree

```

Diapositive 44

```

1 def aux_bytesub(a: str):
2     """Fonction auxiliaire de bytesub qui prend en entrée un élément codé en
3     hexadécimal et renvoie le résultat
4     de la fonction ByteSub appliquée à celui-ci"""
5
6     # on construit b_prime = GF(a)
7     m_gf_i = m_gf_hex_a_dec[a[0]]
8     m_gf_j = m_gf_hex_a_dec[a[1]]
9     b_prime = MATRICE_GF[m_gf_i][m_gf_j]
10    # on construit b à partir de b_prime
11    b_prime = int(b_prime, 16)
12    b_prime = str(bin(b_prime))
13    b_prime = b_prime[2:]
14    b_prime = fu.remplir_zeros(b_prime, 8)
15    b_prime = [[int(b_prime[i])] for i in range(len(b_prime))]
16    b = fu.produit_matriciel(mat_f1, b_prime, (len(mat_f1), len(mat_f1)), (len(
17    b_prime), 1))
18    b = fu.somme_matricielle(b, mat_f2, len(mat_f2), 1)
19    for i in range(len(b)):
20        b[i][0] = b[i][0] % 2
21    b_final = ""
22    for chiffre in b:
23        b_final += str(chiffre[0])
24    b_final = fu.remplir_zeros(b_final, 8)
25    b_final = fu.bin_vers_dec(b_final)
26    b_final = str(hex(b_final))
27
28    return b_final[2:]

```

Diapositive 45

```

1 def bytesub(matrice: list[list]):
2     """Fonction bytesub qui prend en entrée une matrice codée en hexadécimal
3     et renvoie le résultat
4     de la fonction aux_bytesub de AES appliquée à celle-ci"""
5
6     return [[aux_bytesub(matrice[i][j]) for j in range(len(matrice))] for i in
7     range(len(matrice))]
8
9 def shiftrows(matrice: list[list]):
10    """Fonction qui applique la fonction ShiftRows de AES sur la matrice"""
11    n = len(matrice)
12    matrice_finale = []
13    for d in range(n):
14        matrice_finale.append(fu.decale_liste(matrice[d], d))
15
16    return matrice_finale

```

Diapositive 46

```

1 # Algorithme AES implémenté avec une bibliothèque
2 def chiffrer_AES(message:str, cle:bytes, vecteur:bytes):
3     """Fonction qui prend en paramètre un message, une clé et un vecteur et
4     renvoie une chaîne de
5     caractère correspondant au chiffrement de 'message' avec l'algorithme AES
6     en utilisant la clé 'cle'

```

```

5  """et le vecteur d'initialisation, vecteur'\n
6  """-message: chaîne de caractères\n
7  """-cle: chaîne de caractères sur 32 octets\n
8  """-vecteur: chaîne de caractères sur 16 octets"""
9
10     padder = padding.PKCS7(128).padder()
11     padded_data = padder.update(message.encode('utf-8'))
12     padded_data += padder.finalize()
13
14     cipher = Cipher(algorithms.AES(cle), modes.CBC(vecteur), backend=
        default_backend())
15     encryptor = cipher.encryptor()
16     message_chiffre = encryptor.update(padded_data) + encryptor.finalize()
17
18     return message_chiffre

```

Diapositive 47

```

1  def dechiffrer_AES(message_chiffre, cle:bytes, vecteur:bytes):
2      """Fonction qui prend en paramètre un message, une clé et un vecteur et
        renvoie une chaîne de
3  """-caractère correspondant au déchiffrement de 'message_chiffre' avec l'
        algorithme AES en utilisant la clé 'cle'
4  """-et le vecteur d'initialisation, vecteur'\n
5  """-message_chiffre: chaîne de caractères\n
6  """-cle: chaîne de caractères sur 32 octets\n
7  """-vecteur: chaîne de caractères sur 16 octets"""
8
9      decryptor = Cipher(algorithms.AES(cle), modes.CBC(vecteur), backend=
        default_backend()).decryptor()
10     decrypted_data = decryptor.update(message_chiffre) + decryptor.finalize()
11
12     unpadder = padding.PKCS7(128).unpadder()
13     unpadded_data = unpadder.update(decrypted_data)
14     unpadded_data += unpadder.finalize()
15
16     return unpadded_data.decode('utf-8')

```

Diapositive 48

3 serveur.py

```

1  class Serveur:
2      """Classe permettant de simuler un serveur informatique simplifié à l'aide
        du module socket"""
3
4      def __init__(self, modele:int, port=6390):
5          self.host = ""
6          self.port = port
7          self.socket = None
8          self.clients = []
9          self.threads = {}
10         self.id_client = 1
11         self.arret = False
12         self.modele = modele
13
14     def close(self):

```

```

15         """_Fonction_qui_vérifie_si_le_serveur_doit_s'êteindre_ou_non_"""
16         while True:
17             if self.arret == True:
18                 break

```

Diapositive 49

```

1     def connections(self):
2         """_Fonction_qui_gère_les_connexions_au_serveur_d'éventuels_clients_en_
           temps_réel_"""
3         while True:
4             client, ip = self.socket.accept()
5             self.clients.append((self.id_client, client, ip))
6             # on lance un fil d'exécution pour chaque nouveau client afin de
              pouvoir recevoir ses messages
7             # on choisit la bonne fonction de réception en fonction du modèle
              utilisé
8             if self.modele != 3:
9                 self.threads[str(self.id_client)] = threading.Thread(target=
                  self.reception, args=[client, self.id_client])
10            else:
11                self.threads[str(self.id_client)] = threading.Thread(target=
                  self.reception_cm, args=[client, self.id_client])
12            self.threads[str(self.id_client)].start()
13
14            print("Le_client_", self.id_client, "_d'adresse_", ip, "_s'est_
              connecté")
15            self.id_client += 1
16
17        def transfert(self, expéditeur:str, destinataire, message:str, separateur="
          :"):
18            """_Fonction_qui_transfère_un_message_entre_2_clients_"""
19            new_msg = "<Client-" + expéditeur + ">" + separateur + message
20            destinataire.send(new_msg.encode("utf-8"))

```

Diapositive 50

```

1     def reception(self, client, id_client:int):
2         """_Fonction_qui_réceptionne_les_requêtes_des_clients_et_qui_les_traite_
           pour_les_modèles_1_et_2_"""
3         while True:
4             requete_client = client.recv(500)
5             requete_client = requete_client.decode("utf-8")
6             print("<Client-" + str(id_client) + ">:", requete_client)
7             print() # on passe une ligne pour aérer l'affichage
8             if requete_client == "/S_close":
9                 print("CLOSE")
10                self.arret = True
11            else: # on envoie le message au bon destinataire en fonction de l'
                  expéditeur
12                if id_client == 1:
13                    self.transfert(str(id_client), self.clients[1][1],
                      requete_client, separateur="|")
14                else:
15                    self.transfert(str(id_client), self.clients[0][1],
                      requete_client, separateur="|")

```

Diapositive 51

```

1  def reception_cm(self, client, id_client:int):
2      """Fonction qui réceptionne les requêtes des clients et qui les traite
        pour le modèle 3"""
3      while True:
4          requete_client = client.recv(500)
5          requete_client = requete_client.decode("utf-8")
6          decomp_msg = fu.decomposition_chaine(requete_client, separateur="|"
        )
7          print("<Client-" + str(id_client) + ">:", requete_client)
8          print() # on passe une ligne pour aérer l'affichage
9          if requete_client == "/S_close":
10             print("CLOSE")
11             self.arret = True
12          elif "/S_check" in requete_client:
13             if id_client == 1:
14                 destinataire = 1
15             else:
16                 destinataire = 0
17             self.transfert(str(id_client), self.clients[destinataire][1],
                requete_client, separateur="|")

```

Diapositive 52

```

1      elif "/S_init" in requete_client:
2          if len(decomp_msg) > 2: # si le client malveillant veut envoyer
            une clé altérée
3              # on l'envoie au bon destinataire
4              expéditeur = decomp_msg[2]
5              destinataire = int(decomp_msg[3]) - 1
6              # on supprime les infos concernant l'expéditeur et le
            destinataire
7              decomp_msg.pop()
8              decomp_msg.pop()
9              requete_client = fu.compression_arg_str(decomp_msg,
                separateur="|")
10             self.transfert(expéditeur, self.clients[destinataire][1],
                requete_client, separateur="|")
11         else: # sinon c'est la clé d'origine d'un client normal
12             # on envoie donc sa clé au client malveillant
13             self.transfert(str(id_client), self.clients[2][1],
                requete_client, separateur="|")
14         elif id_client != 3: # si le serveur reçoit un message d'un client
            normal
15             # on l'envoie au client malveillant
16             self.transfert(str(id_client), self.clients[2][1],
                requete_client, separateur="|")
17         else:
18             # on l'envoie au bon destinataire
19             expéditeur = decomp_msg[4]
20             destinataire = int(decomp_msg[5]) - 1
21             # on supprime les infos concernant l'expéditeur et le
            destinataire
22             decomp_msg.pop()
23             decomp_msg.pop()
24             requete_client = fu.compression_arg_str(decomp_msg, separateur=
                "|")
25             self.transfert(expéditeur, self.clients[destinataire][1],

```

```
requete_client, separateur="|")
```

Diapositive 53

```
1 def run(self):
2     """Fonction qui démarre et ferme le serveur"""
3     # création du socket pour "démarquer" le serveur
4     self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5     self.socket.bind((self.host, self.port))
6     print("Le serveur est démarré...")
7
8     self.socket.listen(1) # on le met en mode récepteur
9
10    connections = threading.Thread(target=self.connections, args=[])
11    connections.start()
12
13    execution = threading.Thread(target=self.close, args=[])
14    execution.start()
15
16    execution.join()
17
18    for client in self.clients:
19        client[1].close()
20    self.socket.close()
```

Diapositive 54

4 client.py

```
1 class Client_RSA:
2     """Classe permettant de simuler un client pouvant se connecter à un
3     serveur informatique simplifié
4     à l'aide du module socket, qui base le chiffrement sur l'algorithme du RSA
5     """
6
7     def __init__(self, id: int, liste_nb_premiers, host="localhost", port=6390):
8         :
9         self.id = id
10        self.host = host
11        self.port = port
12        self.socket = None
13        self.cle_pv_RSA = None
14        self.cle_pb_RSA = None
15        self.cles_pb_RSA_autres = {}
16        self.liste_nb_premiers = liste_nb_premiers
17        self.init_envoi = False # permet de savoir si self.initialisation_RSA()
18                                # a déjà été exécutée dans self.envoyer()
19        self.init_recu = False # permet de savoir si self.initialisation_RSA()
20                                # a déjà été exécutée dans self.reception()
21        # messages envoyés de la forme : "Client_X:taille_msg:message_chiffre"
22        # On renomme les indices que l'on utilise dans le code pour pouvoir les
23        # modifier plus facilement
24        # et pour plus de visibilité
25        self.i_client = 0
26        self.i_taille_msg = 1
27        self.i_msg_chiffre = 2
```

Diapositive 55

```

1      def initialisation_RSA(self, message: str, separateur="|"):
2          """Fonction qui crée la clé privée et la clé publique du client pour l'
          'algo de chiffrement RSA.
3          Elle est exécutée à l'initialisation de la connection avec le serveur
          et partage la clé publique du client
4          avec un autre client connecté au serveur avec lequel il veut
          communiquer."""
5
6          cles = algos.creation_cles(self.liste_nb_premiers)
7          self.cle_pb_RSA = cles[0]
8          self.cle_pv_RSA = cles[1]
9          msg = fu.compression_arg_str([self.cle_pb_RSA[0], self.cle_pb_RSA[1]])
10         msg = message + separateur + msg
11
12         return msg
13
14     def envoyer(self, socket):
15         """Fonction permettant d'envoyer un message à un client en passant par
          le serveur"""
16         try:
17             while True:
18                 msg = input("-->")
19                 if msg == "/S-init" and not self.init_envoi:
20                     socket.send(self.initialisation_RSA(msg).encode("utf-8"))
21                     self.init_envoi = True
22                 elif msg == "/S-close":
23                     socket.send(msg.encode("utf-8"))

```

Diapositive 56

```

1         else:
2             try:
3                 if self.id == 1: # on chiffre le message avec la clé
4                     # msg_chiffre = algos.chiffrer_RSA(algos.
5                         string_to_int(msg), self.cles_pb_RSA_autres
6                         ["2"])
7                     msg_chiffre = algos.chiffrer_chaine_RSA(msg, self.
8                         cles_pb_RSA_autres["2"], taille_facteur=3)
9                 else:
10                    # msg_chiffre = algos.chiffrer_RSA(algos.
11                        string_to_int(msg), self.cles_pb_RSA_autres
12                        ["1"])
13                    msg_chiffre = algos.chiffrer_chaine_RSA(msg, self.
14                        cles_pb_RSA_autres["1"], taille_facteur=3)
15                # on transforme le message sous forme de liste en
16                chaine de caractères
17                msg_chiffre = fu.compression_arg_str(msg_chiffre)
18                msg_envoye = str(len(msg)) + "|" + str(msg_chiffre) #
19                on ajoute la longueur du message chiffré au message
20                envoyé
21                msg_envoye = msg_envoye.encode("utf-8")
22                socket.send(msg_envoye)
23            except AssertionError:
24                print("Erreur lors de la tentative de chiffrement du
25                    message. Veuillez essayer à nouveau")

```

```

16         except ConnectionResetError:
17             print("Déconnexion du serveur...")
18             sys.exit()

```

Diapositive 57

```

1     def reception(self, socket):
2         """Fonction qui réceptionne les messages reçus du serveur"""
3         try:
4             while True:
5                 requete_serveur = socket.recv(500)
6                 requete_serveur = requete_serveur.decode("utf-8")
7                 # requete_serveur est de la forme : "Client_X:taille_msg: message_chiffre"
8                 decomp_msg = fu.decomposition_chaine(requete_serveur)
9                 id_expediteur = fu.id_client(requete_serveur)
10
11                 if "/S-init" in decomp_msg and not self.init_recu: # si on a
12                     pas enregistré les clés publiques de l'autre client, on le
13                     fait
14                     self.cles_pb_RSA_autres[id_expediteur] = fu.
15                         extraction_arg_str(decomp_msg[self.i_msg_chiffre])
16                     self.init_recu = True
17                     if not self.init_envoi: # si on a pas échangé notre clé
18                         publique, on le fait
19                         socket.send(self.initialisation_RSA("/S-init").encode(
20                             "utf-8"))
21                     self.init_envoi = True

```

Diapositive 58

```

1         else:
2             taille_msg = int(decomp_msg[self.i_taille_msg])
3             # on reconvertit le message chiffré (str) en liste d'
4             entiers pour pouvoir ensuite le déchiffrer
5             msg_chiffre = fu.extraction_arg_str(decomp_msg[self.
6                 i_msg_chiffre])
7             # on vérifie qui est l'expéditeur pour prendre sa clé
8             publique correspondante
9             if id_expediteur == 1:
10                 # msg_dechiffre = algos.dechiffrer_RSA(int(decomp_msg
11                     [2]), self.cle_pv_RSA, self.cle_pb_RSA[0])
12                 msg_dechiffre = algos.dechiffrer_chaine_RSA(msg_chiffre
13                     , self.cle_pv_RSA, self.cle_pb_RSA[0],
14                     taille_entiers=3)
15             else:
16                 msg_dechiffre = algos.dechiffrer_chaine_RSA(msg_chiffre
17                     , self.cle_pv_RSA, self.cle_pb_RSA[0],
18                     taille_entiers=3)
19             print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre
20                 )
21             print("\n-->", end="")
22         except ConnectionResetError:
23             print("Connexion interrompue\nSi le terminal est en attente,
24                 appuyez sur Entrée")
25             sys.exit()

```

Diapositive 59

```

1  def connection(self):
2      """Fonction permettant de connecter le client à un serveur"""
3      self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
4      self.socket.connect((self.host, self.port))
5      envoi = threading.Thread(target=self.envoyer, args=[self.socket])
6      recep = threading.Thread(target=self.reception, args=[self.socket])
7      envoi.start()
8      recep.start()
9
10 class Client_AES_RSA(Client_RSA):
11     """Classe fille de Client_RSA permettant de simuler un client pouvant se
12     connecter à un serveur informatique simplifié
13     à l'aide du module socket, qui base le chiffrement sur une combinaison de l'
14     'algorithme RSA et AES"""
15     def __init__(self, id: int, liste_nb_premiers):
16         Client_RSA.__init__(self, id, liste_nb_premiers)
17         # messages envoyés de la forme : "Client_X/taille_msg/cle_AES/vect_AES/
18         # message_chiffre"
19         # On renomme les indices que l'on utilise dans le code pour pouvoir les
20         # modifier plus facilement
21         # et pour plus de visibilité
22         self.i_client = 0
23         self.i_taille_msg = 1
24         self.i_cle_AES = 2
25         self.i_vect_AES = 3
26         self.i_msg_chiffre = 4

```

Diapositive 60

```

1  def envoyer(self, socket):
2      """Fonction permettant d'envoyer un message à un client en passant par
3      le serveur"""
4      try:
5          while True:
6              msg = input("-->")
7              if msg == "/S_init" and not self.init_envoi:
8                  # on envoie notre clé publique à l'autre client
9                  socket.send(self.initialisation_RSA(msg, separateur="|").
10                     encode("utf-8"))
11                  self.init_envoi = True
12              elif msg == "/S_close":
13                  socket.send(msg.encode("utf-8"))
14              elif msg == "/S_check":
15                  # on construit le hash de la clé publique de l'expéditeur
16                  # et on l'envoie à l'autre client
17                  cle_pb = str(self.cle_pb_RSA[0]) + "/" + str(self.
18                     cle_pb_RSA[1])
19                  hash_cle = hashlib.sha256(cle_pb.encode())
20                  hash_cle = hash_cle.hexdigest()
21                  msg += "|" + hash_cle
22                  socket.send(msg.encode())
23              else:
24                  try:
25                      cle_AES = fu.générateur_alea_chaine(32)
26                      vecteur_AES = fu.générateur_alea_chaine(16)
27                      # si on veut envoyer un msg au client 1, on chiffre
28                      # avec la clé publique du client 2

```

```

24         if self.id == 1: # on chiffre le message avec la clé
25             publique du bon client
                destinataire = "2"

```

Diapositive 61

```

1         else: # sinon c'est l'inverse
2             destinataire = "1"
3             # On chiffre le message avec la clé et le vecteur AES
4             msg_chiffre = algos.chiffrer_AES(msg, fu.encode_ascii(
                    cle_AES), fu.encode_ascii(vecteur_AES))
5             msg_chiffre = msg_chiffre.decode("latin-1")
6             # On chiffre la clé et le vecteur pour AES avec le RSA
7             cle_AES = algos.chiffrer_chaine_RSA(cle_AES, self.
                    cles_pb_RSA_autres[destinataire], taille_facteur=4)
8             vecteur_AES = algos.chiffrer_chaine_RSA(vecteur_AES,
                    self.cles_pb_RSA_autres[destinataire],
                    taille_facteur=4)
9             # on transforme le message, la clé et le vecteur sous
                forme de liste en chaîne de caractères
10            msg_chiffre = fu.compression_arg_str([msg_chiffre])
11            cle_AES = fu.compression_arg_str(cle_AES)
12            vecteur_AES = fu.compression_arg_str(vecteur_AES)
13            # On construit le message que l'on va envoyer contenant
                toutes les infos nécessaires pour le destinataire
14            msg_envoye = str(len(msg)) + "|" + cle_AES + "|" +
                    vecteur_AES + "|" + msg_chiffre
15            msg_envoye = msg_envoye.encode("utf-8")
16            socket.send(msg_envoye)
17        except AssertionError:
18            print("Erreur lors de la tentative de chiffrement du
                    message. Veuillez essayer à nouveau")
19    except ConnectionResetError:
20        print("Déconnexion du serveur...")
21        sys.exit()

```

Diapositive 62

```

1    def reception(self, socket):
2        """Fonction qui réceptionne les messages reçus du serveur"""
3        try:
4            while True:
5                requete_serveur = socket.recv(500)
6                requete_serveur = requete_serveur.decode("utf-8")
7                decomp_msg = fu.decomposition_chaine(requete_serveur,
                    separateur="|")
8                id_expediteur = fu.id_client(requete_serveur)
9                if "/S-init" in decomp_msg and not self.init_recu: # si on a
                    pas enregistré les clés publiques de l'autre client, on le
                    fait
10                   self.cles_pb_RSA_autres[id_expediteur] = fu.
                        extraction_arg_str(decomp_msg[2])
11                   self.init_recu = True
12                   if not self.init_envoi: # si on a pas échangé notre clé
                        publique, on le fait
13                       socket.send(self.initialisation_RSA("/S-init",
                            separateur="|").encode("utf-8"))
14                   self.init_envoi = True

```

```

15         elif "/S-check" in requete_serveur:
16             print("\nComparaison des clés en cours...")
17             # on construit le hash de la clé de l'autre client à partir
               de notre base de donnée
18             cle_pb = str(self.cles_pb_RSA_autres[id_expediteur][0]) + "
               /" + str(self.cles_pb_RSA_autres[id_expediteur][1])
19             hash_cle = hashlib.sha256(cle_pb.encode())
20             hash_cle = hash_cle.hexdigest()
21             # on récupère le hash envoyé par l'autre client
22             hash_autre_cle = decomp_msg[-1]

```

Diapositive 63

```

1             if hash_autre_cle != hash_cle:
2                 print("La communication n'est plus sécurisée !\n")
3             else:
4                 print("La communication est sécurisée")
5         else:
6             # On extrait et déchiffre la clé et le vecteur AES
7             cle_AES = fu.extraction_arg_str(decomp_msg[self.i_cle_AES])
8             cle_AES = algos.dechiffrer_chaine_RSA(cle_AES, self.
               cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
9             vecteur_AES = fu.extraction_arg_str(decomp_msg[self.
               i_vect_AES])
10            vecteur_AES = algos.dechiffrer_chaine_RSA(vecteur_AES, self.
               .cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
11
12            # on récupère le message chiffré et on le déchiffre
13            msg_chiffre = decomp_msg[self.i_msg_chiffre]
14            msg_chiffre = msg_chiffre.encode("latin-1")
15            msg_dechiffre = algos.dechiffrer_AES(msg_chiffre, fu.
               encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
16            print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre
               )
17            print("\n-->", end="")
18        except ConnectionResetError:
19            print("Connexion interrompue\nSi le terminal est en attente,
               appuyez sur Entrée")
20            sys.exit()

```

Diapositive 64

```

1 class ClientMalveillant(Client_AES_RSA):
2     """Classe fille de la classe Client_AES_RSA qui initialise le client
               malveillant voulant lire des conversations
3     sécurisées entre les autres clients connectés au même serveur, dans le
               cadre du modèle n 3 avec la fonction
4     de hachage et le canal secret"""
5     def __init__(self, id: int, liste_nb_premiers):
6         Client_AES_RSA.__init__(self, id, liste_nb_premiers)
7         # Compteur indiquant le nombre d'échanges de clés que le Client
               Malveillant a effectué avec les autres clients
8         self.nb_cles_echangees = 0
9
10    def initialisation_RSA(self, message: str, separateur="|", expéditeur="",
               destinataire=""):
11        """Fonction qui crée la clé privée et la clé publique du client pour l
               'algo de chiffrement RSA.

```

```

12  Elle est exécutée à l'initialisation de la connection avec le serveur
    et partage la clé publique du client
13  avec un autre client connecté au serveur avec lequel il veut
    communiquer. """
14      if self.cle_pv_RSA == None: # si on a pas encore créé nos clés, on le
        fait
15          cles = algos.creation_cles(self.liste_nb_premiers)
16          self.cle_pb_RSA = cles[0]
17          self.cle_pv_RSA = cles[1]
18      msg = fu.compression_arg_str([self.cle_pb_RSA[0], self.cle_pb_RSA[1]])
19      print("clé envoyée:", msg)
20      msg = fu.compression_arg_str([msg, expéditeur, destinataire],
        separateur="|")
21      msg = message + separateur + msg
22      return msg

```

Diapositive 65

```

1      def reception(self, socket):
2          """Fonction qui réceptionne les messages reçus du serveur"""
3          try:
4              while True:
5                  requete_serveur = socket.recv(500)
6                  requete_serveur = requete_serveur.decode("utf-8")
7                  decomp_msg = fu.decomposition_chaine(requete_serveur,
                    separateur="|")
8                  id_expéditeur = fu.id_client(requete_serveur)
9
10                 if "/S-init" in decomp_msg: # si on reçoit une clé publique,
                    on l'enregistre
11                     self.cles_pb_RSA_autres[id_expéditeur] = fu.
                        extraction_arg_str(decomp_msg[2])
12                     print("clé reçue:", fu.extraction_arg_str(decomp_msg[2]))
13                     # on envoie notre propre clé publique au bon destinataire
14                     if id_expéditeur == "1":
15                         socket.send(self.initialisation_RSA("/S-init",
                            separateur="|", expéditeur=id_expéditeur,
                            destinataire="2").encode("utf-8"))
16                     else:
17                         socket.send(self.initialisation_RSA("/S-init",
                            separateur="|", expéditeur=id_expéditeur,
                            destinataire="1").encode("utf-8"))

```

Diapositive 66

```

1      else:
2          # On extrait et déchiffre la clé et le vecteur AES
3          cle_AES = fu.extraction_arg_str(decomp_msg[self.i_cle_AES])
4          cle_AES = algos.dechiffrer_chaine_RSA(cle_AES, self.
            cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
5          vecteur_AES = fu.extraction_arg_str(decomp_msg[self.
            i_vect_AES])
6          vecteur_AES = algos.dechiffrer_chaine_RSA(vecteur_AES, self.
            cle_pv_RSA, self.cle_pb_RSA[0], taille_entiers=4)
7
8          # on récupère le message chiffré et on le déchiffre
9          msg_chiffre = decomp_msg[self.i_msg_chiffre]
10         msg_chiffre = msg_chiffre.encode("latin-1")

```

```

11         msg_dechiffre = algos.dechiffrer_AES(msg_chiffre, fu.
12             encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
13         print("\n", decomp_msg[self.i_client] + ":" + msg_dechiffre
14             )
15         # on renvoie le message au destinataire chiffré avec la clé
16         # publique du destinataire
17         # si on veut envoyer un msg au client 1, on chiffre avec la
18         # clé publique du client 2
19         if id_expediteur == "1": # on chiffre le message avec la
20             # clé publique du bon client
21             destinataire = "2"
22         else: # sinon c'est l'inverse
23             destinataire = "1"

```

Diapositive 67

```

1         # On chiffre le message avec la cle et le vecteur AES
2         msg_chiffre = algos.chiffrer_AES(msg_dechiffre, fu.
3             encode_ascii(cle_AES), fu.encode_ascii(vecteur_AES))
4         msg_chiffre = msg_chiffre.decode("latin-1")
5         # On chiffre la cle et le vecteur pour AES avec le RSA
6         cle_AES = algos.chiffrer_chaine_RSA(cle_AES, self.
7             cles_pb_RSA_autres[destinataire], taille_facteur=4)
8         vecteur_AES = algos.chiffrer_chaine_RSA(vecteur_AES, self.
9             cles_pb_RSA_autres[destinataire], taille_facteur=4)
10        # on transforme le message, la clé et le vecteur sous forme
11        # de liste en chaine de caractères
12        msg_chiffre = fu.compression_arg_str([msg_chiffre])
13        cle_AES = fu.compression_arg_str(cle_AES)
14        vecteur_AES = fu.compression_arg_str(vecteur_AES)
15        # On construit le message que l'on va envoyer contenant
16        # toutes les infos nécessaires pour le destinataire
17        msg_envoye = str(len(msg_dechiffre)) + "|" + cle_AES + "|"
18        + vecteur_AES + "|" + msg_chiffre + "|" + id_expediteur
19        + "|" + destinataire
20        msg_envoye = msg_envoye.encode("utf-8")
21        socket.send(msg_envoye)
22    except ConnectionResetError:
23        print("Connexion interrompue\nSi le terminal est en attente,
24            appuyez sur Entrée")
25        sys.exit()
26
27    def connection(self):
28        """Fonction permettant de connecter le client à un serveur"""
29        self.socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
30        self.socket.connect((self.host, self.port))
31        recep = threading.Thread(target=self.reception, args=[self.socket])
32        recep.start()

```

Diapositive 68

5 fonctions_utiles.py

```

1 def copie_dict(dico:dict):
2     """Fonction qui renvoie une copie indépendante du dictionnaire 'dico'"""
3     dico_copie = {}

```

```

4     for cle, valeur in dico.items():
5         dico_copie[cle] = valeur
6
7     return dico_copie
8
9 def pgcd(a: int, b: int):
10     """Fonction qui prend en paramètre 2 entiers naturels a et b et renvoie
        leur PGCD"""
11
12     dividende = max(a, b)
13     diviseur = min(a, b)
14     temp = 0
15     while (dividende % diviseur) != 0 :
16         temp = dividende % diviseur
17         dividende = diviseur
18         diviseur = temp
19
20     return diviseur

```

Diapositive 69

```

1 def compression_arg_str(liste_args: list, separateur="/"):
2     """Fonction qui prend en paramètre une liste d'objets pouvant être
        convertis en chaîne de caractères, un séparateur '/' et renvoie une
        chaîne de caractères contenant n paramètres et qui est de la forme 'arg
        [1]/arg[2]/.../arg[n]'"""
3     arguments = ""
4     for i in range(len(liste_args)):
5         if i != len(liste_args) - 1:
6             arguments += str(liste_args[i]) + separateur
7         else:
8             arguments += str(liste_args[i])
9     return arguments
10
11 def extraction_arg_str(chaine_args: str, separateur="/"):
12     """Fonction qui prend en paramètre une chaîne de caractères contenant n
        arguments et de la forme 'arg[1]/arg[2]/.../arg[n]', un séparateur '/'
        et renvoie une liste contenant les n arguments de type int"""
13     chiffres = "0123456789"
14     liste_args = []
15     arg_courant = ""
16     for i in range(len(chaine_args)):
17         if chaine_args[i] != separateur and chaine_args[i] in chiffres:
18             arg_courant += chaine_args[i]
19         else:
20             liste_args.append(int(arg_courant))
21             arg_courant = ""
22     if arg_courant != "":
23         liste_args.append(int(arg_courant))
24     return liste_args

```

Diapositive 70

```

1 def decomposition_chaine(chaine: str, separateur="|"):
2     """Fonction qui prend en paramètre une chaîne de caractères que l'on peut
        diviser en sous-chaînes séparées par un séparateur ':' et renvoie une
        liste contenant ces sous-chaînes"""
3     liste_sous_chaines = []

```

```

4     arg_courant = ""
5     for i in range(len(chaine)):
6         if chaine[i] != separateur:
7             arg_courant += chaine[i]
8         else:
9             liste_sous_chaines.append(arg_courant)
10            arg_courant = ""
11    if arg_courant != "":
12        liste_sous_chaines.append(arg_courant)
13    return liste_sous_chaines
14
15    def id_client(chaine: str):
16        """Fonction qui prend une chaîne de caractères de type '<Client-*>***',
17           et renvoie l'entier * en chaîne de caractères"""
18        chiffres = "0123456789"
19        id_passe = False
20        id = ""
21        for char in chaine:
22            if char in chiffres:
23                id_passe = True
24                id += char
25            elif id_passe:
26                break
27        return id

```

Diapositive 71

```

1    def importer_fichier(nom_fichier: str, type="int"):
2        """Importe un fichier texte et ajoute chaque ligne du fichier dans une
3           liste Python que la fonction renvoie"""
4        liste_nb_premiers = []
5        with open(nom_fichier, "r") as fichier:
6            contenu = fichier.readlines()
7        for ligne in contenu:
8            ligne_actuelle = ligne.rstrip("\n")
9            if type == "int":
10                liste_nb_premiers.append(int(ligne_actuelle))
11            else:
12                liste_nb_premiers.append(ligne_actuelle)
13        return liste_nb_premiers
14
15    def generateur_alea_chaine(taille_chaine: int):
16        """Fonction qui génère aléatoirement une chaîne de caractères de taille
17           taille_chaine, composée des lettres de l'alphabet en minuscule, et la
18           renvoie"""
19        liste_int_a_char = "abcdefghijklmnopqrstuvwxyz"
20        nb_char = len(liste_int_a_char)
21        chaine_alea = ""
22        for _ in range(taille_chaine):
23            i = rd.randint(0, nb_char - 1)
24            chaine_alea += liste_int_a_char[i]
25        return chaine_alea

```

Diapositive 72

```

1    def concat_chaine(chaine: str, n: int):
2        """Fonction qui renvoie la concaténation de 'chaine' (n-1) fois"""
3

```

```

4     chaine_finale = ""
5     for _ in range(n):
6         chaine_finale += chaine
7
8     return chaine_finale
9
10 def encode_ascii(chaine: str):
11     """_Fonction qui transforme une chaine_str en bstr (ie en code ASCII)\n
12     NB: elle fait la même chose que la fonction native bytes(chaine, encoding=
13         "ascii")_"""
14     return bytes(chaine, encoding="utf-8")

```

Diapositive 73

```

1 def matrice_gf(chaine: str, taille=16):
2     """_Renvoie la matrice GF sous forme de matrice de type list[list]_"""
3     matrice_gf = decomposition_chaine(chaine, separateur="_")
4     matrice_gf_bis = []
5     matrice_gf_finale = []
6     ligne_gf = []
7
8     for i in range(len(matrice_gf)):
9         if "\n" in matrice_gf[i]:
10             matrice_gf_bis.append(matrice_gf[i][:-1])
11         elif matrice_gf[i] != "":
12             matrice_gf_bis.append(matrice_gf[i])
13
14     for i in range(taille):
15         for j in range(taille):
16             ligne_gf.append(matrice_gf_bis[i*taille + j])
17             if j == taille - 1:
18                 matrice_gf_finale.append(ligne_gf)
19                 ligne_gf = []
20
21     return matrice_gf_finale

```

Diapositive 74

```

1 def conversion_matrice(matrice: list[list], fonction, base=10):
2     """_Fonction qui applique la fonction de conversion 'fonction' sur chaque
3         case de la matrice carrée_"""
4     res = matrice.copy()
5     for i in range(len(matrice)):
6         for j in range(len(matrice)):
7             if base != 10:
8                 res[i][j] = fonction(res[i][j], base)
9             else:
10                 res[i][j] = fonction(res[i][j])
11
12     return res

```

Diapositive 75

```

1 def produit_matriciel(m1: list[list[int]], m2: list[list[int]], n1=(-1, -1), n2
2     =(-1, -1)):
3     """_Fonction qui renvoie le produit matriciel des matrices m1 et m2_"""
4     assert(n1[1] == n2[0])

```

```

4     if n1 == (-1, -1):
5         n = len(m1)
6         p = n
7         q = n
8     else:
9         n, p = n1[0], n1[1]
10        q = n2[1]
11    res = [[0 for _ in range(q)] for _ in range(n)]
12    for i in range(n):
13        for j in range(q):
14            somme = 0
15            for k in range(p):
16                somme += m1[i][k] * m2[k][j]
17            res[i][j] = somme
18    return res
19
20 def somme_matricielle(m1: list[list[int]], m2: list[list[int]], n: int, p: int)
21 :
22     """Fonction qui renvoie la somme des matrices m1 et m2"""
23     res = [[0 for _ in range(p)] for _ in range(n)]
24     for i in range(n):
25         for j in range(p):
26             res[i][j] = m1[i][j] + m2[i][j]
27     return res

```

Diapositive 76

```

1  def bin_vers_dec(x: str):
2      """Fonction qui convertit un élément de type int binaire en élément de
3         type int décimal"""
4      x_str = str(x)
5      x_liste = [int(c) for c in x_str]
6      x_liste.reverse()
7      x_int = 0
8      k = 0
9      for c in x_liste:
10         x_int += 2**k * c
11         k += 1
12
13     return x_int
14
15 def remplir_zeros(x, n: int):
16     """Rajoute des zéros jusqu'à ce que 'x' soit un nombre binaire codé sur n
17        bits"""
18     x_str = str(x)
19     while len(x_str) < n:
20         x_str = "0" + x_str
21
22     return x_str

```

Diapositive 77

```

1  def affiche_matrice(m: list[list]):
2      """Fonction qui affiche une matrice carrée"""
3      for i in range(len(m)):
4          for j in range(len(m)):
5              print(m[i][j], " ", end="")
6          print()

```

```

7
8 def decale_liste(liste: list, d: int):
9     """Fonction qui décale les éléments de la liste de 'd' vers la droite
10    et renvoie le résultat"""
11     liste_bis = liste.copy()
12     liste_finale = []
13     d_bis = d
14
15     if d == 0:
16         return liste
17
18     while d_bis > 0:
19         liste_finale = []
20         liste_finale.append(liste_bis.pop())
21         for el in liste_bis:
22             liste_finale.append(el)
23         liste_bis = liste_finale.copy()
24         d_bis -= 1
25
26     return liste_finale

```

Diapositive 78